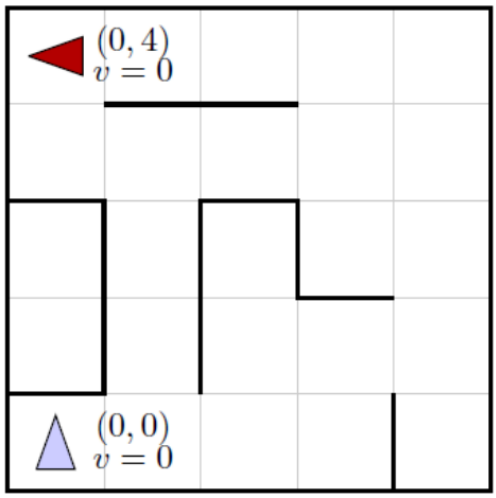
BÀI TẬP TÌM KIẾM VÀ HEURISTIC TRONG AI

Họ và tên	Lê Minh Hiếu
MSSV / Lớp	3124720023

Mục tiêu học tập

- Mô hình hóa bài toán tìm kiếm trong AI.
- Vận dụng BFS, UCS, Greedy và A*.
- Phân tích và đánh giá heuristic.
- Phân biệt heuristic admissible và consistent.
- So sánh tính đầy đủ, tối ưu và hiệu quả của các thuật toán tìm kiếm.

Câu 1. Tìm kiếm và heuristic



Một tác tử dạng xe cần thoát khỏi mê cung như hình trên. Tác tử có hướng và tại mọi thời điểm luôn quay về một trong bốn hướng d ∈ {Bắc, Nam, Đông, Tây}. Trong mỗi bước, tác tử có thể tiến về phía trước với vận tốc có thể điều chỉnh hoặc thực hiện thao tác rẽ.

Các hành động di chuyển gồm tăng tốc, giữ tốc độ và giảm tốc. Sau hành động, tác tử di chuyển một số ô bằng vận tốc mới v'. Gọi v là vận tốc hiện tại và v' là vận tốc sau điều chỉnh:

- Tăng tốc (Faster): v' = v + 1
- Giảm tốc (Slower): v' = v - 1
- Giữ tốc độ (Maintain): v' = v

Các hành động rẽ gồm trái và phải, làm thay đổi hướng 90°. Chỉ được phép rẽ khi vận tốc bằng 0; sau khi rẽ vận tốc vẫn bằng 0.

- Rẽ trái: quay 90° ngược chiều kim đồng hồ.
- Rẽ phải: quay 90° theo chiều kim đồng hồ.

Ví dụ: nếu tác tử ở (0,0), quay về Bắc, v = 0 và cần đến (2,0), quay về Đông, v = 0 thì một chuỗi hành động hợp lệ là: rẽ phải, tăng tốc, giữ tốc độ, giảm tốc.

Các hành động không hợp lệ gồm:

- Hành động làm tác tử va vào tường.
- Làm $v < 0$ hoặc $v > Vmax$.
- Giữ tốc độ khi $v = 0$.
- Rẽ khi $v \neq 0$.

Mục tiêu là tìm kế hoạch đưa xe dừng hẳn ($v = 0$) tại ô thoát, đúng hướng đích, với số hành động ít nhất. Chi phí đường đi được tính bằng số hành động đã thực hiện.

a) Giả sử tác tử đi theo nhánh ngoài cùng bên trái (đi qua ô (1,2)), từ trạng thái bắt đầu (0,0), hướng Bắc đến trạng thái đích (0,4), hướng Tây. Hãy viết chuỗi hành động ngắn nhất.

Chuỗi hành động (15 hành động): Rẽ phải → Tăng tốc → Giảm tốc → Rẽ trái → Tăng tốc → Giữ tốc độ → Giữ tốc độ → Giảm tốc → Rẽ trái → Tăng tốc → Giảm tốc → Rẽ phải → Tăng tốc → Giảm tốc → Rẽ trái.

Chi phí = 15. Lưu ý: không thể đi (1,0)→(1,3) bằng "Tăng tốc, Tăng tốc, Giảm tốc, Giảm tốc" vì lần giảm tốc đầu (v'=1) sẽ đi thêm 1 ô và đâm vào tường phía trên (1,3); đoạn 3 ô ngắn nhất là Tăng tốc, Giữ, Giữ, Giảm tốc (4 hành động). Phía Bắc (0,0) có tường nên bắt buộc phải rẽ phải ngay từ đầu.

Bước	Hành động	Trạng thái sau (x, y, hướng, v)
0	Trạng thái bắt đầu	(0,0, Bắc, 0)
1	Rẽ phải	(0,0, Đông, 0)
2	Tăng tốc	(1,0, Đông, 1)
3	Giảm tốc	(1,0, Đông, 0)
4	Rẽ trái	(1,0, Bắc, 0)
5	Tăng tốc	(1,1, Bắc, 1)
6	Giữ tốc độ	(1,2, Bắc, 1)
7	Giữ tốc độ	(1,3, Bắc, 1)
8	Giảm tốc	(1,3, Bắc, 0)
9	Rẽ trái	(1,3, Tây, 0)
10	Tăng tốc	(0,3, Tây, 1)
11	Giảm tốc	(0,3, Tây, 0)

Bước	Hành động	Trạng thái sau (x, y, hướng, v)
12	Rẽ phải	(0,3, Bắc, 0)
13	Tăng tốc	(0,4, Bắc, 1)
14	Giảm tốc	(0,4, Bắc, 0)
15	Rẽ trái	(0,4, Tây, 0) ← đích

(Trạng thái ghi theo dạng (x, y, hướng, v) sau mỗi hành động.)

b) Nếu lưới có kích thước M × N và vận tốc tối đa là Vmax, kích thước không gian trạng thái là bao nhiêu? Giả sử mọi cấu hình đều có thể đạt được từ trạng thái bắt đầu.

Kích thước không gian trạng thái = 4 · M · N · (Vmax + 1)

Trạng thái = (x, y, hướng, v): M·N vị trí × 4 hướng × (Vmax + 1) giá trị vận tốc v ∈ {0, 1, …, Vmax}.

c) Một "nút con" của trạng thái s là trạng thái s' có thể đạt được bằng một hành động hợp lệ từ s. Có thể tồn tại trạng thái không có nút con hay không? Nếu có, hãy cho ví dụ; nếu không, hãy giải thích.

☒ Có ☐ Không

Ví dụ: s = (0, 0, Bắc, v = 2) (với Vmax ≥ 2) – phía trước ngay sát là tường.

- Tăng tốc (v' = 3, đi 3 ô) → đâm tường.
- Giữ tốc độ (v' = 2, đi 2 ô) → đâm tường.
- Giảm tốc (v' = 1, đi 1 ô) → vẫn đâm tường (0,1).
- Rẽ trái / rẽ phải → không hợp lệ vì v ≠ 0.

⇒ s không có nút con. Tổng quát: mọi trạng thái có v ≥ 2 và tường ngay phía trước đều không có nút con.

d) Hệ số phân nhánh lớn nhất của bài toán là bao nhiêu? Hãy cho một trạng thái ví dụ (x, y, hướng, vận tốc) đạt hệ số phân nhánh này và liệt kê các hành động có thể thực hiện.

Hệ số phân nhánh lớn nhất b = 3.

Có 5 hành động nhưng chúng loại trừ nhau theo v: v = 0 → chỉ {Tăng tốc, Rẽ trái, Rẽ phải}; 0 < v < Vmax → {Tăng tốc, Giữ, Giảm tốc}; v = Vmax → {Giữ, Giảm tốc}. Không bao giờ có quá 3 hành động hợp lệ.

Ví dụ: (1, 0, Đông, 0) → các hành động: Tăng tốc (đến (2,0), v = 1), Rẽ trái (hướng Bắc, v = 0), Rẽ phải (hướng Nam, v = 0).

(Một ví dụ khác với v > 0: (1, 1, Bắc, 1), Vmax ≥ 2 → Tăng tốc, Giữ tốc độ, Giảm tốc.)

e) Khoảng cách Manhattan từ vị trí hiện tại của tác tử đến vị trí cửa ra có phải là heuristic chấp nhận được không? Nếu không, hãy cho một trạng thái phản ví dụ, nêu giá trị heuristic và chi phí thực tế tối ưu đến đích.

☐ Có ☒ Không

Không admissible, vì mỗi hành động có thể đi nhiều ô (v' > 1) nên số hành động có thể ít hơn khoảng cách Manhattan.

Phản ví dụ: hành lang thẳng không có tường, Vmax ≥ 2; trạng thái (0, 0, Đông, 0), cửa ra tại (6, 0) hướng Đông.

- h = |6 − 0| + |0 − 0| = 6.
- Chi phí tối ưu = 5: Tăng tốc (v=1, +1 ô) → Tăng tốc (v=2, +2 ô) → Giữ tốc độ (+2 ô) → Giảm tốc (v=1, +1 ô) → Giảm tốc (v=0, +0 ô): tổng 6 ô, 5 hành động.

⇒ h = 6 > h* = 5: heuristic đánh giá quá cao.

Ghi chú: *trong mê cung 5×5 ở hình, đoạn thẳng dài nhất chỉ 4 ô nên khó dựng phản ví dụ ngay trên hình này; nhưng với lưới M × N tổng quát (đề hỏi) thì kết luận là Không.*

f) Heuristic h = khoảng cách Manhattan / Vmax có phải là admissible không? Giải thích. Nếu không, hãy cho phản ví dụ tương tự câu (e).

☒ Có ☐ Không

Mỗi hành động di chuyển tối đa Vmax ô nên làm khoảng cách Manhattan giảm tối đa Vmax. Để giảm khoảng cách D về 0 cần ít nhất D / Vmax hành động (chưa tính các hành động rẽ/dừng bắt buộc) ⇒ h(s) ≤ h*(s) với mọi s, và h(đích) = 0. Do đó h admissible.

Kiểm tra lại phản ví dụ ở (e): h = 6 / 2 = 3 ≤ 5 ✓.

g) Nếu dùng heuristic không admissible trong A* Tree Search, tính đầy đủ của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.

☐ Có ☒ Không

Chi phí mỗi hành động ≥ 1 > 0 nên g(n) tăng không giới hạn dọc theo mọi nhánh vô hạn (do các chu trình), còn h hữu hạn. Chỉ có hữu hạn nút có f = g + h ≤ f(nút đích), nên A* chắc chắn sẽ mở rộng đích sau hữu hạn bước ⇒ vẫn complete (chỉ có thể mất tính tối ưu).

h) Nếu dùng heuristic không admissible trong A* Tree Search, tính tối ưu của thuật toán có thể thay đổi không? Giả sử đồ thị hữu hạn và heuristic không âm.

☒ Có ☐ Không

Có thể mất tối ưu. Ví dụ: S→G chi phí 10; S→A chi phí 1; A→G chi phí 1. Đặt h(S) = 0, h(G) = 0, h(A) = 100 (đánh giá quá cao).

Hàng đợi sau khi mở S: G có f = 10, A có f = 1 + 100 = 101 → A* mở G trước và trả về S→G (chi phí 10), trong khi đường tối ưu S→A→G có chi phí 2.

i) Những lý do nào sau đây có thể hợp lý khi dùng heuristic không admissible thay vì admissible? Chọn tất cả phương án phù hợp.

☒ Heuristic không admissible có thể dễ tính hơn, giúp giảm thời gian tính heuristic.

☒ Heuristic không admissible có thể gần chi phí thực hơn (dù đôi khi đánh giá cao), nhờ đó có thể mở ít nút hơn.

☐ Heuristic không admissible vẫn luôn tìm được đường đi tối ưu khi mọi chi phí thực không âm.

☒ Heuristic không admissible có thể được dùng để làm thuật toán hoàn toàn không xét một phần của đồ thị.

Chọn phương án 1, 2 và 4. Phương án 3 sai (ví dụ ở câu h). Phương án 4: h rất lớn ở một vùng khiến f = g + h của vùng đó luôn lớn nên không bao giờ được mở – tăng tốc nhưng có thể đánh đổi tính tối ưu.

Câu 2. Các nút tìm kiếm

Xét tree search (không dùng tập explored) cho một bài toán tìm kiếm bất kỳ có hệ số phân nhánh tối đa b. Mỗi nút tìm kiếm n có chi phí tích lũy g(n), heuristic admissible h(n) và độ sâu d(n). Gọi n_c là nút đích có chi phí nhỏ nhất và n_s là nút đích nông nhất.

Với mỗi trường hợp, hãy viết một bất đẳng thức mô tả tập các nút được mở rộng trước khi thuật toán kết thúc. Không cần xử lý trường hợp hòa (tie). Nếu không có nút nào thỏa, ghi "không có".

a) Breadth-First Search (BFS).

Bất đẳng thức: d(n) < d(n_s)

BFS mở rộng theo từng tầng nên mọi nút nông hơn nút đích nông nhất đều được mở rộng trước khi tìm ra đích.

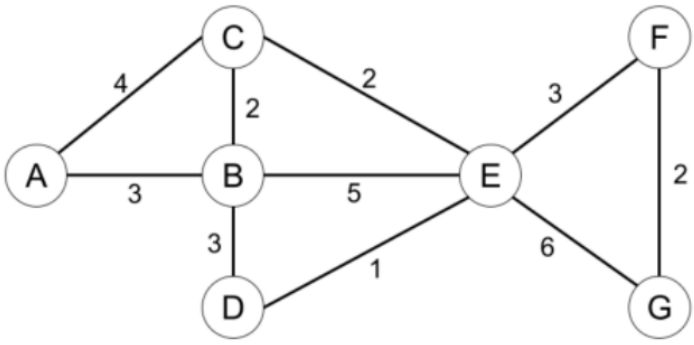
b) Uniform-Cost Search (UCS).

> **Bất đẳng thức: g(n) < g(n_c)**
> UCS mở rộng theo thứ tự g tăng dần nên mọi nút có chi phí tích lũy nhỏ hơn chi phí của đích tối ưu đều được mở rộng.

c) A* Search, giả sử h là heuristic nhất quán.

> **Bất đẳng thức: g(n) + h(n) < g(n_c) (tức f(n) < f(n_c))**
> A* mở rộng theo f = g + h tăng dần; tại đích h(n_c) = 0 nên f(n_c) = g(n_c).

Câu 3. Tìm kiếm trên đồ thị



Nút	h1	h2
A	12	11
B	6	7
C	9	6
D	3	4
E	3	5
F	2	1
G	0	0

Xét đồ thị vô hướng như hình. A là trạng thái bắt đầu và G là trạng thái đích. Chi phí của mỗi cạnh được ghi trên đồ thị. Bảng trên cung cấp hai heuristic h1 và h2. Khi có nhiều nút cùng ưu tiên, kết quả có thể phụ thuộc quy tắc phá hòa (tie-breaking).

a) Với mỗi chiến lược tìm kiếm đồ thị dưới đây, hãy đánh dấu X vào mọi đường đi mà thuật toán có thể trả về. Nếu kết quả không nằm trong các đường đi liệt kê, ghi đường đi đúng ở cột "Khác".

Thuật toán	A-C-E-G	A-C-E-F-G	A-B-D-E-F-G	Khác
UCS	☐	☒	☐	☐
Greedy với h1	☐	☐	☐	☐ A-B-E-G (hoặc A-B-D-E-G, tùy phá hòa)
Greedy với h2	☒	☐	☐	☐
A* với h1	☐	☐	☒	☐
A* với h2	☐	☒	☐	☐

Chi phí các đường: A-C-E-G = 12; A-C-E-F-G = 11 (tối ưu); A-B-D-E-F-G = 12; A-B-E-G = 14; A-B-D-E-G = 13.

- **UCS:** luôn trả đường tối ưu A-C-E-F-G.
- **Greedy h1:** A(12) → mở A, có B(6), C(9) → mở B → có D(3), E(3) (hòa) → khi mở E thì G(0) vào hàng đợi và được lấy ngay (h = 0) ⇒ A-B-E-G (hoặc A-B-D-E-G nếu E được thêm lại từ D và chọn bản đó).
- **Greedy h2:** A → C(6) → E(5) → G(0) ⇒ A-C-E-G.
- **A* h1 (f = g + h1):** mở A(12), B(9), D(9), E(10, g = 7), F(12), rồi G (g = 12, f = 12) ⇒ A-B-D-E-F-G (chi phí 12, không tối ưu).
- **A* h2 (f = g + h2):** B và C cùng f = 10 (hòa), nhưng dù thứ tự nào, E được mở với g = 6 (qua C), F(f = 10), rồi G (f = 11) ⇒ A-C-E-F-G.

b) Chi phí của đường đi tối ưu từ A đến G theo UCS là bao nhiêu?

Đáp án: 11 (đường A-C-E-F-G: 4 + 2 + 3 + 2)

c) Heuristic h1 có admissible không? Có consistent không?

Admissible: ☐ Có ☒ Không Consistent: ☐ Có ☒ Không

Chi phí thực tối ưu đến G: h*(A)=11, h*(B)=9, h*(C)=7, h*(D)=6, h*(E)=5, h*(F)=2, h*(G)=0.

h1 không admissible: h1(A) = 12 > 11 và h1(C) = 9 > 7. Không consistent (consistent ⇒ admissible); ví dụ cạnh C–E: h1(C) = 9 > c(C,E) + h1(E) = 2 + 3 = 5.

d) Heuristic h2 có admissible không? Có consistent không?

Admissible: ☒ Có ☐ Không Consistent: ☐ Có ☒ Không

h2 admissible: h2(A)=11 ≤ 11, h2(B)=7 ≤ 9, h2(C)=6 ≤ 7, h2(D)=4 ≤ 6, h2(E)=5 ≤ 5, h2(F)=1 ≤ 2, h2(G)=0.

Không consistent: cạnh A–C: h2(A) = 11 > c(A,C) + h2(C) = 4 + 6 = 10 (cạnh A–B cũng vi phạm: 11 > 3 + 7 = 10).

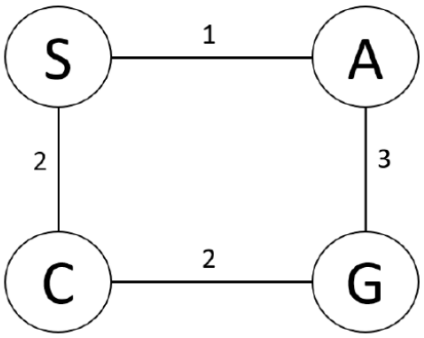
Câu 4. Trắc nghiệm và trả lời ngắn về tìm kiếm

a) Đánh dấu Đúng/Sai. Giả sử mọi hành động có chi phí ít nhất ε, với ε > 0, và mọi heuristic được dùng đều consistent.

Mệnh đề	Đúng	Sai	Giải thích ngắn
Depth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.	☐	☒	Tree search không có tập explored nên DFS có thể lặp vô hạn trên chu trình.

Mệnh đề	Đúng	Sai	Giải thích ngắn
Breadth-First Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.	☒	☐	BFS duyệt theo tầng, nếu có đích ở độ sâu d sẽ tìm thấy.
Iterative Deepening Tree Search trên một đồ thị hữu hạn luôn được đảm bảo là complete.	☒	☐	Giới hạn độ sâu tăng dần nên không bị kẹt trong chu trình và sẽ chạm tới độ sâu của đích.
Với mọi đồ thị không có chu trình, Graph Search luôn có frontier lớn hơn Tree Search.	☐	☒	Graph Search chỉ nhỏ hơn hoặc bằng; nếu đồ thị là cây thì hai cách như nhau.
Iterative Deepening Graph Search có độ phức tạp thời gian của BFS và độ phức tạp bộ nhớ của DFS.	☐	☒	Graph Search phải lưu tập explored nên bộ nhớ không còn như DFS (tính chất này chỉ đúng với Tree Search).
Nếu h1(s) và h2(s) đều là heuristic consistent thì min(h1(s), h2(s)) cũng nhất thiết consistent.	☒	☐	Với h = min(h1,h2): $h(s) \le h_i(s) \le c + h_i(s')$; chọn i đạt min tại s' ⇒ $h(s) \le c + h(s')$.

b) Xét đồ thị trạng thái dưới đây, S là bắt đầu và G là đích. Hãy điền các giá trị heuristic sao cho heuristic admissible nhưng không consistent.



Trạng thái	h(s)
S	4
A	2
C	2
G	0

Chi phí thực tối ưu đến G: $h^*(S) = 4$ (S-A-G hoặc S-C-G), $h^*(A) = 3$, $h^*(C) = 2$, $h^*(G) = 0$.

Admissible: $h(S)=4 \leq 4$, $h(A)=2 \leq 3$, $h(C)=2 \leq 2$, $h(G)=0$ ✓.

Không consistent: cạnh S–A: $h(S) = 4 > c(S,A) + h(A) = 1 + 2 = 3$. (Các giá trị khác thỏa, ví dụ $h(A) = 0$ hoặc 1 cũng được.)

--- Hết ---